

Integrationstests mit Pipelines und Jobs

Section 5 · Implementing CI-CD · Quelle: Kurs 4, M02 Kapitel 12–13 + tests/integration_test/

Unit-Tests prüfen einzelne Funktionen isoliert – sie sagen aber nichts darüber aus, ob eine komplette Pipeline mit ihren echten Ein- und Ausgabedaten korrekt funktioniert. Dafür braucht es **Integrationstests**. Für Lakeflow Declarative Pipelines (SDP) bieten sich zwei komplementäre Ansätze an: Datenqualitäts-Expectations direkt in der Pipeline, und separate Validierungs-Tasks in einem Lakeflow Job.

1. Integrationstests über SDP-Expectations

Eine einfache und wirkungsvolle Methode ist, innerhalb der Pipeline selbst eine Materialized View zu definieren, die eine erwartete Kennzahl (z. B. Zeilenanzahl je Umgebung) berechnet und über eine Expectation gegen einen Zielwert prüft. Da sich die erwartete Zeilenanzahl zwischen Entwicklungs- und Staging-Umgebung unterscheidet, wird der Zielwert dynamisch anhand der Pipeline-Konfiguration `target` ausgewählt:

```

from pyspark import pipelines as dp

target = spark.conf.get("target")

# Erwartete Zeilenzahlen je Umgebung (aus dem Kursbeispiel)
target_integration_tests_validation = {
    'development': {'health_bronze': {'total_rows': 7500}, 'health_silver': {'total_rows': 7500}},
    'stage':        {'health_bronze': {'total_rows': 35000}, 'health_silver': {'total_rows': 35000}}
}

if target in ('development', 'stage'):
    total_expected_bronze = target_integration_tests_validation[target]['health_bronze']['total_rows']
    total_expected_silver = target_integration_tests_validation[target]['health_silver']['total_rows']

def test_count_table_total_rows(table_name, total_count, target):
    """Erstellt eine Materialized View, die die Zeilenanzahl einer Tabelle
    gegen den für die aktuelle Umgebung erwarteten Wert prüft."""
    @dp.expect_all_or_fail({"row_count_matches": f"count = {total_count}"})
    @dp.table(name=f"test_{table_name}_row_count")
    def _row_count_check():
        return spark.sql(f"SELECT COUNT(*) AS count FROM {table_name}")

```

Schlägt die Expectation fehl (z. B. weil nach einer Code-Änderung plötzlich Zeilen verloren gehen), bricht der Pipeline-Lauf ab bzw. wird die Abweichung im Event-Log sichtbar – je nachdem, welche der drei Expectation-Aktionen (WARN/DROP ROW/FAIL UPDATE) gewählt wurde. In Produktion wird meist bewusst nur ein Teil der Tests ausgeführt (z. B. nur die Gold-Tabellen-Prüfung), um die Laufzeit gering zu halten.

2. Integrationstests über Lakeflow-Jobs-Tasks

Alternativ oder ergänzend lässt sich ein mehrstufiger Lakeflow Job aufbauen, dessen Tasks nacheinander ausgeführt werden: zuerst ein Task, der die Unit-Tests via `pytest` ausführt, danach ein Pipeline-Task für die eigentliche SDP-ETL-Pipeline, und optional ein abschließender Visualisierungs- oder Validierungs-Task. Schlägt ein früherer Task fehl, werden nachfolgende Tasks (dank Task-Abhängigkeiten, siehe Section 4) gar nicht erst gestartet – fehlerhafter Code gelangt so nicht in die nächste Umgebung.

Ergänzt aus der offiziellen Databricks-Dokumentation: Databricks empfiehlt, für das Testen von Pipelines gezielt eigene Entwicklungs-/Test-Datasets mit sowohl korrekten als auch bewusst fehlerhaften Datensätzen zu verwenden, um zu prüfen, ob die Pipeline-Logik (inklusive Expectations) Fehlerfälle wie erwartet behandelt. Da jeder Pipeline-Lauf ein strukturiertes Event-Log als Delta-Tabelle schreibt (mit Ausführungsfortschritt, Expectation-Ergebnissen und Datenherkunft), kann dieses Log direkt für automatisierte Integrationstests über dev/stage/prod hinweg abgefragt werden, statt eigene Zähl-Views wie oben von Hand zu pflegen.

Quelle: [Manage data quality with pipeline expectations – Databricks-Dokumentation](#)